Cheatsheet · 9 chapters on one page

Data Structures and Algorithms cheatsheet

Every formula, key idea, table, memory aid and common mistake from the chapters, gathered in one place for quick revision. Nothing here is new. Each card links back to the lesson that teaches it, so open the lesson when a line does not make sense yet.

106 key ideas | 91 formulas | 41 tables | 165 tips and traps

STEP 1 Getting started

1.1 Introduction to DSA

A program is two things working together: a way to organise data, and a recipe that works on it. Choose them well and a task that would take hours finishes in a blink. This page builds the whole picture, from the first definitions to a method you can use on any problem.

KEY IDEAS

- **What data structures and algorithms are** A data structure is a way of organising data in memory so that the operations you need are easy and fast. A list of numbers kept in order, a stack of undo steps and a map from names to phone numbers are all data structures.
- **Why DSA matters** Computers are fast, roughly a hundred million to a billion simple steps per second. That speed hides bad choices on small inputs and exposes them on large ones.
- **What makes something an algorithm** Donald Knuth listed five properties that every algorithm must have.
- **Kinds of data structures** Both hold items in a line. They differ only in which end you remove from, and that one difference makes them fit completely different jobs.
- **Abstract data types** An abstract data type (ADT) describes what a type does: its values and its operations, with the rules they follow. It says nothing about how those operations are built. A data structure is one concrete way to build an ADT.
- **Core operations and what they cost** Almost every task is a mix of a few basic operations: access (read by position), search (find by value), insert , delete , traverse (visit everything), sort and merge . Each structure makes some of these cheap and others expensive.
- **Algorithm design paradigms** Most algorithms are built from a small set of strategies. Recognising which one fits is half of solving a problem.
- **Choosing the right structure**
- **A method for solving any problem**

TABLES

Table: input size n · n steps · n $\log_2$ n steps · n^2 steps

Input size n	n steps	n $\log_2$ n steps	n^2 steps
1,000	1 µs	10 µs	1 ms
1,000,000	1 ms	20 ms	about 17 minutes
1,000,000,000	1 s	30 s	about 32 years

Lesson: Why DSA matters

Table: property · meaning · breaks it

Property	Meaning	Breaks it
Finiteness	It always stops after a finite number of steps.	"Keep adding 1 until the number is negative."
Definiteness	Every step is precise, with exactly one meaning.	"Add a pinch of salt."
Input	It takes zero or more well-defined inputs.	Using data that was never given.
Output	It produces at least one result related to the input.	A procedure that computes but reports nothing.
Effectiveness	Each step is basic enough to be done exactly, in finite time.	"Write down every digit of π."

Lesson: What makes something an algorithm

Table: grouping · meaning · examples

Grouping	Meaning	Examples
Primitive	Built into the language, holds a single value	int, float, char, boolean
Linear	Elements form a sequence, each has at most one next	array, linked list, stack, queue
Non-linear	An element can connect to several others	tree, heap, graph, trie
Hash based	A function maps each key to a slot	hash table, hash set
Static	Size fixed when created	array in C or Java
Dynamic	Grows and shrinks while running	Python list, ArrayList, linked list

Lesson: Kinds of data structures

Table: adt · operations · common implementations

ADT	Operations	Common implementations
List	get(i), insert(i, x), remove(i), size	dynamic array, linked list
Stack	push, pop, peek, isEmpty	dynamic array, linked list
Queue	enqueue, dequeue, front, isEmpty	circular buffer, linked list, two stacks
Map (dictionary)	put(k, v), get(k), remove(k)	hash table, balanced search tree
Set	add, contains, remove	hash table, balanced search tree, bitset
Priority queue	insert, peekMin, removeMin	binary heap

Lesson: Abstract data types

Table: structure · access · search · insert

Structure	Access	Search	Insert	Delete
Array (sorted)	O(1)	O(log n) binary search	O(n)	O(n)
Dynamic array	O(1)	O(n)	O(1) amortized at the end, O(n) in the middle	O(1) at the end, O(n) in the middle
Singly linked list	O(n)	O(n)	O(1) at the head or after a known node	O(1) at the head
Stack	top only, O(1)	O(n)	push O(1)	pop O(1)
Queue	front only, O(1)	O(n)	enqueue O(1)	dequeue O(1)
Hash table	by key	O(1) average, O(n) worst	O(1) average	O(1) average
Balanced search tree	by key	O(log n)	O(log n)	O(log n)
Binary heap	min or max only, O(1)	O(n)	O(log n)	remove min O(log n)

Lesson: Core operations and what they cost

Table: paradigm · idea · classic examples

Paradigm	Idea	Classic examples
Brute force	Try every candidate	checking all pairs, linear search
Greedy	Take the best looking choice now and never undo it	Dijkstra's shortest paths, Huffman coding, activity selection
Divide and conquer	Split into independent parts, solve each, combine	merge sort, quick sort, binary search
Dynamic programming	Solve overlapping subproblems once and reuse the answers	coin change, knapsack, longest common subsequence
Backtracking	Build a solution step by step and undo a step that cannot lead anywhere	N-Queens, Sudoku, generating subsets
Two pointers and sliding window	Move two indexes cleverly instead of nesting loops	pair sum on sorted data, longest substring without repeats
Graph traversal	Explore vertices in a planned order	breadth first search, depth first search

Lesson: Algorithm design paradigms

Table: if you mostly need to... · reach for

If you mostly need to...	Reach for
Read or write by position, with a known or growing size	array or dynamic array
Find, add or remove by key quickly	hash map or hash set
Keep keys sorted and ask for the next or previous one	balanced search tree (TreeMap, SortedList)
Undo, match brackets, or go back the way you came	stack
Process things in arrival order	queue
Always take the smallest or largest next	heap (priority queue)
Insert or remove next to a known element very often	linked list
Model connections: roads, friends, dependencies	graph
Look up words by prefix	trie

Lesson: Choosing the right structure

Table: stage · topics

Stage	Topics
1. Foundations	this introduction, run time analysis, recursion
2. Linear structures	arrays, strings, hashing, linked lists, stacks, queues
3. Core techniques	sorting, binary search, two pointers, sliding window, prefix sums
4. Non-linear structures	trees, binary search trees, heaps, tries, graphs
5. Advanced paradigms	greedy, divide and conquer, dynamic programming, backtracking

Lesson: A method for solving any problem

TIPS AND TRAPS

REMEMBER

The two always come as a pair. Binary search is fast only because the data is kept **sorted in an array**, where any position can be reached in one step.

Lesson: What data structures and algorithms are

IN THE EXAM

Technical interviews test DSA because it shows whether you can reason about correctness and cost, not just make code run.

Lesson: Why DSA matters

REMEMBER **Why is it correct?** Before each new comparison, `best` is the largest of the values seen so far. This statement is called a **loop invariant**. It is true at the start (one value seen), each step keeps it true, and when the loop ends "seen so far" means "all values".

Lesson: What makes something an algorithm

COMMON MISTAKE In JavaScript, `array.shift()` removes the first element by moving every other one, which is O(n). The example keeps an index to the front instead, so each removal is O(1).

Lesson: Kinds of data structures

REMEMBER Program against the ADT, then pick the data structure for the costs you need. Java's `List` interface with `ArrayList` or `LinkedList` behind it is exactly this idea.

Lesson: Abstract data types

REMEMBER There is no best data structure. There is only the best one for the operations your program does most often.

Lesson: Core operations and what they cost

COMMON MISTAKE With coins 1, 3 and 4, greedy takes 4 first and then needs two 1s. Dynamic programming discovers that 3 + 3 uses only two coins. A greedy idea must be proved before it is trusted.

Lesson: Algorithm design paradigms

REMEMBER The input size tells you which complexity you can afford. With n up to 10^5, an $O(n^2)$ solution does about 10^{10} steps, which is too slow, so look for O(n log n) or O(n).

Lesson: A method for solving any problem

1.2 Run time analysis

Two programs can give the same answer while one finishes instantly and the other runs for years. Run time analysis predicts which is which before you run anything, for time and for memory. By the end you will read a loop or a recursive function and state its cost with confidence.

KEY IDEAS

- **Why count steps instead of timing** Timing a program with a stopwatch tells you about one machine, one language, one input and one moment. Change any of these and the number changes. We want a measure that belongs to the algorithm itself .
- **Orders of growth** From best to worst: $1 < \log n < \sqrt{n} < n < n \log n < n^2 < n^3 < 2^n < n!$
- **Big O, Big Omega and Big Theta** Asymptotic notation compares how two functions grow when n becomes large. Here f(n) is the step count and g(n) is a simple reference such as n^2.
- **Rules for finding the complexity** The triangle loop does $1 + 2 + \ldots + (n - 1) = n(n - 1)\ 2 = 64 \times 63\ 2 = 2016$ steps. Half of n^2 is still $O(n^2)$.
- **Best, worst, average and amortized** Some operations are usually cheap and occasionally expensive. Amortized analysis spreads the rare expensive cost over the whole sequence.
- **Space complexity** Space complexity measures how the memory an algorithm uses grows with n. It has two parts:
- **Auxiliary space in depth** When someone asks for the space complexity of an algorithm, they almost always mean its auxiliary space . The input has to be stored anyway, so including it would make every algorithm at least O(n) and hide the real differences.
- **Recurrences and the recursion tree** A recursive function's cost is described by a recurrence : the cost for n written in terms of the cost for smaller inputs. For merge sort:
- **Recursion depth and the call stack** Every function call pushes a stack frame holding its parameters, local variables and the place to return to. The frame is popped when the call returns.
- **From constraints to complexity** A rough rule: a typical judge or server does about 10^8 simple steps per second. The input limits then tell you which complexity is acceptable for a one second limit.

FORMULAS

Why count steps instead of timing

Exam favourite

$$T(n) = n + 2$$

Lesson: Why count steps instead of timing

Orders of growth

Exam favourite

$$1 < \log n < \sqrt{n} < n < n \log n < n^2 < n^3 < 2^n < n!$$

Lesson: Orders of growth

Orders of growth

$$\log_2 n = \log_{10} n \div \log_{10} 2$$

Lesson: Orders of growth

Big O, Big Omega and Big Theta

Exam favourite

$$3n^2 + 5n + 2 = O(n^2)$$

Lesson: Big O, Big Omega and Big Theta

Big O, Big Omega and Big Theta

$$3n^2 + 5n + 2 \le 4n^2 \iff n^2 - 5n - 2 \ge 0$$

Lesson: Big O, Big Omega and Big Theta

Rules for finding the complexity

Exam favourite

$$1 + 2 + \ldots + (n-1) = \frac{n(n-1)}{2} = \frac{64 \times 63}{2} = 2016$$

Lesson: Rules for finding the complexity

Best, worst, average and amortized

Exam favourite

$$1 + 2 + 4 + \ldots < 2n$$

Lesson: Best, worst, average and amortized

Space complexity

Exam favourite

Total space = input space + auxiliary space

Lesson: Space complexity

Auxiliary space in depth

Exam favourite

$$\log_2 16 + 1 = 5$$

Lesson: Auxiliary space in depth

Recurrences and the recursion tree

Exam favourite

$$T(n) = 2T(n/2) + n,\ T(1) = 1$$

Lesson: Recurrences and the recursion tree

Recurrences and the recursion tree

$$T(n) = n + n + \ldots + n\ (\log_2 n \text{ levels}) = n \log_2 n = \Theta(n \log n)$$

Lesson: Recurrences and the recursion tree

Recurrences and the recursion tree

$$T(n) = a \cdot T(n/b) + f(n),\ a \ge 1,\ b > 1$$

Lesson: Recurrences and the recursion tree

Recurrences and the recursion tree

n^(log_b a)

Lesson: Recurrences and the recursion tree

TABLES

Table: name · order · typical example

Name	Order	Typical example
Constant	O(1)	read a[i], push on a stack, hash lookup on average
Logarithmic	O(log n)	binary search, one step down a balanced tree
Square root	O(√n)	trial division up to √n
Linear	O(n)	linear search, sum of an array
Linearithmic	O(n log n)	merge sort, heap sort, good comparison sorting
Quadratic	O(n²)	all pairs, bubble sort, insertion sort worst case
Cubic	O(n³)	naive matrix multiplication, Floyd-Warshall
Exponential	O(2ⁿ)	all subsets, plain recursive Fibonacci
Factorial	O(n!)	all orderings (permutations)

Lesson: Orders of growth

Table: notation · meaning · formal definition

Notation	Meaning	Formal definition
$f(n) = O(g(n))$	upper bound: f grows no faster than g	there exist $c > 0$ and n_0 with $0 \le f(n) \le c \cdot g(n)$ for all $n \ge n_0$
$f(n) = \Omega(g(n))$	lower bound: f grows at least as fast as g	there exist $c > 0$ and n_0 with $f(n) \ge c \cdot g(n) \ge 0$ for all $n \ge n_0$
$f(n) = \Theta(g(n))$	tight bound: same growth	both $O(g(n))$ and $\Omega(g(n))$: $c_1 \cdot g(n) \le f(n) \le c_2 \cdot g(n)$ for all $n \ge n_0$
$f(n) = o(g(n))$	strictly slower growth	$f(n) \div g(n) \to 0$ as $n \to \infty$
$f(n) = \omega(g(n))$	strictly faster growth	$f(n) \div g(n) \to \infty$ as $n \to \infty$

Lesson: Big O, Big Omega and Big Theta

Table: rule · example · result

Rule	Example	Result
Drop constant factors	5n, n/2, 100n	$O(n)$
Drop lower order terms	$n^2 + 3n + 7$	$O(n^2)$
Sequential blocks add, the biggest wins	$O(n)$ loop, then $O(n^2)$ loop	$O(n + n^2) = O(n^2)$
Nested blocks multiply	loop of n containing a loop of m	$O(n \cdot m)$
If and else: take the more expensive branch	if: $O(1)$, else: $O(n)$	$O(n)$ in the worst case
Different inputs keep different letters	loop over a (size n) then b (size m)	$O(n + m)$, not $O(n)$

Lesson: Rules for finding the complexity

Table: pattern · iterations · order

Pattern	Iterations	Order
`for i in range(n)`	n	$O(n)$
two nested loops over n	n^2	$O(n^2)$
inner loop starts at `i + 1`	$n(n - 1) \div 2$	$O(n^2)$
`i *= 2` or `i //= 2`	about $\log_2 n$	$O(\log n)$
`while i * i <= n`	about $\sqrt{n}$	$O(\sqrt{n})$
loop over n containing a halving loop	about $n \log_2 n$	$O(n \log n)$

Lesson: Rules for finding the complexity

Table: case · question it answers · linear search on n items

Case	Question it answers	Linear search on n items
Best	the luckiest input of size n	target first: 1 check, $\Theta(1)$
Worst	the unluckiest input of size n	target last or missing: n checks, $\Theta(n)$
Average	the expected cost over a stated distribution of inputs	$(n + 1) \div 2$ checks if every position is equally likely, $\Theta(n)$

Lesson: Best, worst, average and amortized

Table: approach · time · auxiliary space

Approach	Time	Auxiliary space
Compare every pair	$O(n^2)$	$O(1)$
Sort first, then check neighbours (in place heap sort)	$O(n \log n)$	$O(1)$
Hash set of seen values	$O(n)$ average	$O(n)$

Lesson: Space complexity

Table: sorting algorithm · time (worst) · auxiliary space · in place

Sorting algorithm	Time (worst)	Auxiliary space	In place
Bubble, insertion, selection	$O(n^2)$	$O(1)$	yes
Heap sort	$O(n \log n)$	$O(1)$	yes
Quick sort	$O(n^2)$, $O(n \log n)$ average	$O(\log n)$ stack on average, $O(n)$ worst	yes, by the relaxed definition
Merge sort (arrays)	$O(n \log n)$	$O(n)$ buffer	no
Counting sort	$O(n + k)$	$O(n + k)$	no

Lesson: Auxiliary space in depth

Table: recurrence · typical algorithm · solution

Recurrence	Typical algorithm	Solution
$T(n) = T(n - 1) + 1$	factorial, walking a linked list recursively	$\Theta(n)$
$T(n) = T(n/2) + 1$	binary search	$\Theta(\log n)$
$T(n) = 2T(n/2) + 1$	visiting every node of a balanced tree	$\Theta(n)$
$T(n) = 2T(n/2) + n$	merge sort	$\Theta(n \log n)$
$T(n) = T(n - 1) + n$	selection sort written recursively	$\Theta(n^2)$
$T(n) = 2T(n - 1) + 1$	towers of Hanoi, all subsets	$\Theta(2^n)$
$T(n) = T(n - 1) + T(n - 2) + 1$	plain recursive Fibonacci	$\Theta(\varphi^n)$, $\varphi \approx 1.618$

Lesson: Recurrences and the recursion tree

Table: case · condition · result · example

Case	Condition	Result	Example
1. Leaves dominate	f(n) grows polynomially slower than n^(log_b a)	Θ(n^(log_b a))	$T(n) = 4T(n/2) + n$ gives $\Theta(n^2)$
2. Balanced	f(n) = Θ(n^(log_b a))	Θ(n^(log_b a) · log n)	merge sort: Θ(n log n). Binary search: Θ(log n)
3. Root dominates	f(n) grows polynomially faster, and a·f(n/b) ≤ k·f(n) for some k < 1	Θ(f(n))	$T(n) = T(n/2) + n$ gives $\Theta(n)$

Lesson: Recurrences and the recursion tree

Table: function · total calls (time) · depth (stack space)

Function	Total calls (time)	Depth (stack space)
factorial(n)	n	n, so O(n)
plain Fibonacci	about 2×1.618^n	n, so O(n)
memoized Fibonacci	about 2n	n, plus O(n) for the memo
binary search	about $\log_2 n$	about $\log_2 n$
merge sort	about 2n	about $\log_2 n$ (plus the O(n) buffer)
all subsets of n items	$2^{n+1} - 1$	n + 1

Lesson: Recursion depth and the call stack

Table: language · what happens · typical limit

Language	What happens	Typical limit
Python	`RecursionError: maximum recursion depth exceeded`	1000 frames by default (`sys.getrecursionlimit()`)
Java	`java.lang.StackOverflowError`	depends on the thread stack size (set with `-Xss`), often several thousand to tens of thousands of frames
JavaScript	`RangeError: Maximum call stack size exceeded`	depends on the engine, often around ten thousand frames

Lesson: Recursion depth and the call stack

Table: largest n · acceptable complexity · typical technique

Largest n	Acceptable complexity	Typical technique
about 10	O(n!)	try all permutations
about 20 to 25	$O(2^n)$, $O(n \cdot 2^n)$	all subsets, bitmask DP
about 500	$O(n^3)$	triple loops, Floyd-Warshall
about 5,000 to 10,000	$O(n^2)$	all pairs, simple DP tables
about 10^6	O(n log n)	sorting, heaps, binary search on the answer
about 10^8	O(n)	one pass, two pointers, prefix sums
larger	O(log n) or O(1)	maths formulas, binary search, fast power

Lesson: From constraints to complexity

TIPS AND TRAPS

REMEMBER

Run time analysis asks one question: **as n grows, how fast does the number of steps grow?** Constants and small terms are ignored because they do not change that answer.

Lesson: Why count steps instead of timing

REMEMBER

The base of the logarithm does not matter inside Big O, because

$$\log_2 n = \log_{10} n \div \log_{10} 2$$

and dividing by a constant is ignored. Computer scientists usually mean $\log_2$.

Lesson: Orders of growth

COMMON MISTAKE

A common mix-up. Big O is not the same thing as "worst case". Big O is an upper bound on a function. The worst case running time, the best case running time and the average running time are three different functions, and each one can be described with O, Ω or Θ.

Lesson: Big O, Big Omega and Big Theta

IN THE EXAM

If an algorithm is O(n), it is also $O(n^2)$ and $O(2^n)$, since those are still upper bounds. When people say "the complexity is O(n)" they mean the tightest bound, which is really Θ(n).

Lesson: Big O, Big Omega and Big Theta

REMEMBER

The **worst case** is the most useful by default. It is a guarantee: the algorithm never does worse, whatever input arrives.

Lesson: Best, worst, average and amortized

REMEMBER

Amortized is not the same as average case. Average case depends on random inputs. Amortized cost is guaranteed for every sequence of operations.

Lesson: Best, worst, average and amortized

REMEMBER An algorithm is **in place** when its auxiliary space is O(1), or O(log n) by some definitions that allow a small recursion stack.

Lesson: Auxiliary space in depth

REMEMBER Karatsuba multiplication, T(n) = 3T(n/2) + n, falls in case 1: Θ(n^(log$_2$ 3)) ≈ Θ(n^1.585). That beats the school method's Θ(n²).

Lesson: Recurrences and the recursion tree

REMEMBER Time counts **all** the calls. Auxiliary space for recursion counts only the calls that are active **at the same time**: the depth.

Lesson: Recursion depth and the call stack

REMEMBER **Tail recursion** means the recursive call is the very last action. Some languages reuse the frame for it, but Python and Java never do, and among JavaScript engines only Safari does. Do not rely on it. Write the loop yourself.

Lesson: Recursion depth and the call stack

COMMON MISTAKE A recursive factorial or list walk on a million items will crash with a stack overflow, even though its time is fine. Convert deep linear recursion into a loop, which needs O(1) stack space.

Lesson: Recursion depth and the call stack

STEP 2 Core building blocks

2.1 Mathematics in DSA

22 maths problems from primes and GCD to Miller-Rabin, naive versus optimized, with quizzes, number practice and a six language narrator.

KEY IDEAS

- **Primality Testing** √N bound with 6k ± 1 candidates: O(√N), against O(N) for try every d from 2 to n − 1.
- **GCD and LCM (Euclidean Algorithm)** Euclidean algorithm: O(log min(a, b)), against O(min(a, b)) for count down from min(a, b).
- **Counting Digits and Palindrome Number** Arithmetic with half reversal: O(d), against O(d) for convert to a string.
- **All Divisors of N** Pairs up to √n: O(√n), against O(n) for test every d from 1 to n.
- **Power of Two and Counting Set Bits** Kernighan and n & (n − 1): O(set bits) / O(1), against O(32) / O(log n) for scan all 32 bits / divide by 2.
- **Base Conversion (Any Base 2 to 36)** Repeated division and Horner: O(log_b n), against O(b · log_b n) for greedy subtraction of powers.
- **Nth Fibonacci Number** Iterate with two variables: O(n), against O(φⁿ) ≈ O(1.618ⁿ) for plain recursion.
- **Sieve of Eratosthenes** Sieve of Eratosthenes: O(N log log N), against O(N √N) for trial-divide each number.
- **Modular Exponentiation (Fast Power)** Binary exponentiation: O(log exp), against O(exp) for repeated multiplication.
- **Prime Factorization** Trial division to √n: O(√n), against O(n) for prime n for divide by every d until n = 1.
- **Binomial Coefficient nCr mod p** Factorials and Fermat inverses: O(n + log p) build, O(1) query.
- **Integer Square Root** Binary search on the answer: O(log n), against O(√n) for count up.
- **Inclusion-Exclusion: Count Multiples** Inclusion-exclusion over subsets: O(2^k · k log N), against O(N · k) for check every x in 1 … n.
- **Modular Inverse (Extended Euclid)** Extended Euclidean algorithm: O(log m), against O(m) for try every x.
- **Matrix Exponentiation (Nth Fibonacci in O(log N))** Matrix binary exponentiation: O(log n) (8 mults per step), against O(n) for iterate n steps.

- **Catalan Numbers** Ratio from the closed form: O(n) big-number steps, against about O(3ⁿ) for plain recursion.
- **Chinese Remainder Theorem** Pairwise merging with Extended Euclid: O(k log M), against O(lcm · k) for search 0 … lcm − 1.
- **Segmented Sieve (Primes in a Range)** Segmented sieve: O((R − L) log log R + √R), against O((R − L) √R) for trial-divide each number in [l, r].
- **Miller-Rabin Primality Test** Deterministic Miller-Rabin: O(12 · log³ n), against O(√n) for trial division to √n.

FORMULAS

Primality Testing

Exam favourite

$$N = a \times b \;\Rightarrow\; \min(a, b) \le \sqrt{N}$$

Lesson: Primality Testing

Primality Testing

$$p > 3 \text{ prime} \;\Rightarrow\; p \equiv \pm 1 \pmod 6$$

Lesson: Primality Testing

GCD and LCM (Euclidean Algorithm)

Exam favourite

$$\gcd(a, b) = \gcd(b, \mathbf{a \bmod b}), \quad \gcd(a, 0) = |a|$$

Lesson: GCD and LCM (Euclidean Algorithm)

GCD and LCM (Euclidean Algorithm)

$$\operatorname{lcm}(a, b) = |a| / \gcd(a, b) \times |b|$$

Lesson: GCD and LCM (Euclidean Algorithm)

Counting Digits and Palindrome Number

Exam favourite

$$\text{digit} = n \bmod 10, \quad n = \lfloor n / 10 \rfloor$$

Lesson: Counting Digits and Palindrome Number

Counting Digits and Palindrome Number

$$\text{rev} = \text{rev} \times 10 + \text{digit}$$

Lesson: Counting Digits and Palindrome Number

Counting Digits and Palindrome Number

$$\text{digits}(n) = \lfloor \log_{10}|n| \rfloor + 1 \quad (n \neq 0)$$

Lesson: Counting Digits and Palindrome Number

All Divisors of N

Exam favourite

$$i \mid n \;\Leftrightarrow\; (n / i) \mid n, \quad \min(i, n / i) \le \sqrt{\mathbf{n}}$$

Lesson: All Divisors of N

All Divisors of N

$$d(n) = (e_1+1)(e_2+1)\ldots \text{ for } n = p_1^{e_1}p_2^{e_2}\ldots$$

Lesson: All Divisors of N

Power of Two and Counting Set Bits

Exam favourite

$x \;\&\; (x - 1)$ clears the **lowest set bit**

Lesson: Power of Two and Counting Set Bits

Power of Two and Counting Set Bits

n is a power of two $\;\Leftrightarrow\;$ $n > 0$ and $n \;\&\; (n - 1) = 0$

Lesson: Power of Two and Counting Set Bits

Base Conversion (Any Base 2 to 36)

Exam favourite

$$n = d_k b^k + \ldots + d_1 b + d_0$$

Lesson: Base Conversion (Any Base 2 to 36)

Base Conversion (Any Base 2 to 36)

$$d_0 = n \bmod b, \quad n \leftarrow \lfloor n / b \rfloor$$

Lesson: Base Conversion (Any Base 2 to 36)

Base Conversion (Any Base 2 to 36)

$$\text{value} = (\ldots((d_k)b + d_{k-1})b + \ldots)b + d_0$$

Lesson: Base Conversion (Any Base 2 to 36)

Nth Fibonacci Number

Exam favourite

$$F(n) = F(n - 1) + F(n - 2)$$

Lesson: Nth Fibonacci Number

Nth Fibonacci Number

$$\text{calls}(n) = 2 \cdot F(n + 1) - 1 \approx \mathbf{1.618^n}$$

Lesson: Nth Fibonacci Number

Nth Fibonacci Number

$$(a, b) \leftarrow (b, a + b)$$

Lesson: Nth Fibonacci Number

Sieve of Eratosthenes

Exam favourite

for p = 2 … √N: if p unmarked, mark **p^2, $p^2 + p$, …**

Lesson: Sieve of Eratosthenes

Sieve of Eratosthenes

$$\Sigma\ N/p \text{ over primes } p \le N \approx N \cdot \mathbf{\ln \ln N}$$

Lesson: Sieve of Eratosthenes

Sieve of Eratosthenes

$$\pi(N) \approx N / \ln N \text{ primes up to } N$$

Lesson: Sieve of Eratosthenes

Modular Exponentiation (Fast Power)

Exam favourite

$$a^e = (a^2)^{e/2} \text{ if e is even}, \quad a \cdot a^{e-1} \text{ if e is odd}$$

Lesson: Modular Exponentiation (Fast Power)

Modular Exponentiation (Fast Power)

$$(x \cdot y) \bmod m = ((x \bmod m) \cdot (y \bmod m)) \bmod m$$

Lesson: Modular Exponentiation (Fast Power)

Modular Exponentiation (Fast Power)

$$\text{steps} = \lfloor \log_2 e \rfloor + 1$$

Lesson: Modular Exponentiation (Fast Power)

Factorial and Trailing Zeros

Exam favourite

$$\text{zeros}(n!) = \min(v_2, v_5) = v_5(n!)$$

Lesson: Factorial and Trailing Zeros

Factorial and Trailing Zeros

$$v_5(n!) = \lfloor n/5 \rfloor + \lfloor n/25 \rfloor + \lfloor n/125 \rfloor + \dots$$

Lesson: Factorial and Trailing Zeros

Factorial and Trailing Zeros

$$n! = 1 \times 2 \times \dots \times n, \quad 0! = 1$$

Lesson: Factorial and Trailing Zeros

Prime Factorization

Exam favourite

$$n = p_1^{e_1} \cdot p_2^{e_2} \cdot \dots \cdot p_k^{e_k}$$

Lesson: Prime Factorization

Prime Factorization

loop while **$d \cdot d \le n$**, then leftover $n > 1$ is prime

Lesson: Prime Factorization

Prime Factorization

SPF query: $n \rightarrow n / \text{spf}[n]$, at most $\log_2 n$ steps

Lesson: Prime Factorization

Exam favourite

$$\varphi(n) = n \cdot \prod_{p \mid n} (1 - \mathbf{1/p})$$

Lesson:

$$\varphi(p) = p - 1, \quad \varphi(p^k) = p^k - p^{k-1}$$

Lesson:

$$a^{\varphi(n)} \equiv 1 \pmod{n} \text{ when } \gcd(a, n) = 1$$

Lesson:

Binomial Coefficient nCr mod p

Exam favourite

$$C(n, r) = C(n-1, r-1) + C(n-1, r)$$

Lesson: Binomial Coefficient nCr mod p

Binomial Coefficient nCr mod p

$$C(n, r) = n! \cdot \mathbf{(r!)^{-1}} \cdot \mathbf{((n-r)!)^{-1}} \bmod p$$

Lesson: Binomial Coefficient nCr mod p

Binomial Coefficient nCr mod p

$$x^{-1} \equiv x^{p-2} \pmod{p}, \text{ p prime}$$

Lesson: Binomial Coefficient nCr mod p

Integer Square Root

Exam favourite

$$x = \max\{\ k : k^2 \le n\ \}$$

Lesson: Integer Square Root

Integer Square Root

$$\text{mid} \le \lfloor n / \text{mid} \rfloor \ \Leftrightarrow\ \text{mid}^2 \le n$$

Lesson: Integer Square Root

Integer Square Root

$$\text{Newton: } x_{k+1} = \lfloor (x_k + \lfloor n / x_k \rfloor) / 2 \rfloor$$

Lesson: Integer Square Root

Inclusion-Exclusion: Count Multiples

Exam favourite

$$|A_1 \cup \ldots \cup A_k| = \Sigma_{S \neq \varnothing} (-1)^{|S|+1} \lfloor N / \mathrm{lcm}(S) \rfloor$$

Lesson: Inclusion-Exclusion: Count Multiples

Inclusion-Exclusion: Count Multiples

$$|A \cup B| = |A| + |B| - |A \cap B|$$

Lesson: Inclusion-Exclusion: Count Multiples

Modular Inverse (Extended Euclid)

Exam favourite

$$a \cdot x + m \cdot y = \gcd(a, m) \ \text{(Bezout)}$$

Lesson: Modular Inverse (Extended Euclid)

Modular Inverse (Extended Euclid)

$$\gcd = 1 \ \Rightarrow \ a \cdot x \equiv 1 \pmod m \ \Rightarrow \ \mathbf{a^{-1} = x \bmod m}$$

Lesson: Modular Inverse (Extended Euclid)

Modular Inverse (Extended Euclid)

$$\text{back-substitution: } x = y', \quad y = x' - \lfloor A/B \rfloor \cdot y'$$

Lesson: Modular Inverse (Extended Euclid)

Matrix Exponentiation (Nth Fibonacci in O(log N))

Exam favourite

$$[[1, 1], [1, 0]]^n = [[F(n+1), \mathbf{F(n)}], [F(n), F(n-1)]]$$

Lesson: Matrix Exponentiation (Nth Fibonacci in O(log N))

Matrix Exponentiation (Nth Fibonacci in O(log N))

$$M^n = (M^2)^{n/2} \ \text{ or } \ M \cdot M^{n-1}$$

Lesson: Matrix Exponentiation (Nth Fibonacci in O(log N))

Matrix Exponentiation (Nth Fibonacci in O(log N))

$$(XY)_{ij} = \Sigma_k X_{ik} Y_{kj} \bmod m$$

Lesson: Matrix Exponentiation (Nth Fibonacci in O(log N))

Exam favourite

$$n = \Sigma\, n_i\, p^i, \quad r = \Sigma\, r_i\, p^i$$

Lesson:

$$C(n, r) \equiv \mathbf{\prod_i C(n_i, r_i)} \pmod p$$

Lesson:

$$r_i > n_i \text{ for some } i \ \Rightarrow \ C(n, r) \equiv 0$$

Lesson:

Catalan Numbers

Exam favourite

$$C(0) = 1, \quad C(n) = \Sigma_{i=0}^{n-1} C(i) \cdot C(n - 1 - i)$$

Lesson: Catalan Numbers

Catalan Numbers

$$C(n) = \mathbf{C(2n, n) / (n + 1)}$$

Lesson: Catalan Numbers

Catalan Numbers

$$C(i + 1) = C(i) \cdot 2(2i + 1) / (i + 2)$$

Lesson: Catalan Numbers

Chinese Remainder Theorem

Exam favourite

$$x = \mathrm{cur} + M \cdot t, \quad M \cdot t \equiv a - \mathrm{cur} \pmod m$$

Lesson: Chinese Remainder Theorem

Chinese Remainder Theorem

$$g = \gcd(M, m), \quad \text{need } g \mid (a - \mathrm{cur})$$

Lesson: Chinese Remainder Theorem

Chinese Remainder Theorem

$$t = (a - \mathrm{cur})/g \cdot (M/g)^{-1} \bmod (m/g), \quad M \leftarrow \mathbf{lcm(M, m)}$$

Lesson: Chinese Remainder Theorem

Segmented Sieve (Primes in a Range)

Exam favourite

$$\mathrm{start}(p) = \max(p^2, \lceil L / p \rceil \cdot p)$$

Lesson: Segmented Sieve (Primes in a Range)

Segmented Sieve (Primes in a Range)

$$\text{cross start, start + p, } \ldots \le R$$

Lesson: Segmented Sieve (Primes in a Range)

Segmented Sieve (Primes in a Range)

$$\text{memory: } O(\sqrt{R} + R - L + 1)$$

Lesson: Segmented Sieve (Primes in a Range)

Miller-Rabin Primality Test Exam favourite

$$n - 1 = d \cdot 2^s, \quad d \text{ odd}$$

Lesson: Miller-Rabin Primality Test

Miller-Rabin Primality Test

n prime $\Rightarrow$ $a^d \equiv 1$ or $a^{d \cdot 2^r} \equiv -1$ for some $0 \le r < s$

Lesson: Miller-Rabin Primality Test

Miller-Rabin Primality Test

bases $\{2, 3, 5, \ldots, 37\}$ are exact for $n < 3.3 \times 10^{24}$

Lesson: Miller-Rabin Primality Test

2.2 Arrays

32 lessons from memory layout to segment trees, including heap sort, Tim sort, shell, counting, radix, bucket and cycle sort, 128 practice problems, quizzes, number practice and a six language narrator.

KEY IDEAS

- **Memory Allocation: Stack vs Heap Stack allocation**: memory is reserved by moving the stack pointer. It is very fast and is released automatically when the function returns. Size must be small and is usually known in advance (C: `int a[5]`).
- **Contiguous Memory Layout** An array is one unbroken block of bytes. Element `i` starts at byte offset `i × size`.
- **Cache Locality Spatial locality**: if you touch address *x*, you will probably touch *x+1* soon. Hardware loads whole cache lines (usually 64 bytes, or 16 ints).
- **Array Indexing Mechanics 0-based**: C, C++, Java, JavaScript, Python, Go, Rust. The index equals the offset in elements, so no extra subtraction is needed.
- **Static Arrays Compile-time size**: the length is a constant in the source code (C: `int a[10]`). The compiler can reserve it on the stack or in static storage.
- **Dynamic Arrays and Amortized Cost** Keeps **size** (used slots) and **capacity** (allocated slots). Push writes at `size` when `size < capacity`.
- **Insertion, Deletion and Shifting Insert at i**: shift `a[i..n−1]` right by one, starting from the end so nothing is overwritten. Cost O(n − i).
- **Multi-Dimensional Arrays Row-major**: C, C++, NumPy default (`order='C'`). **Column-major**: Fortran, MATLAB, R, Julia, NumPy `order='F'`.
- **Jagged Arrays** Also called an **array of arrays** or **array of pointers**. The outer array stores references, each row is a separate heap block.
- **Linear Search** Works on **unsorted** data. No setup needed.
- **Binary Search** Needs a **sorted** array. Time O(log n), space O(1) when written as a loop.
- **Bubble, Insertion and Selection Sort** All three are **in-place** (O(1) extra space) and $O(n^2)$ in the worst case.
- **Merge Sort and Quick Sort Merge sort**: O(n log n) always, stable, needs O(n) extra space for the temporary array. Great for linked lists and external sorting.
- **Heap Sort** A **binary heap** is stored in the array itself: the children of index i are

$$2i + 1$$

and

$$2i + 2$$

, its parent is

$$\lfloor (i - 1) / 2 \rfloor$$

.

- **TimSort and Introsort (Hybrid Sorts) TimSort** (Python `sorted`, Java `Arrays.sort` for objects, V8 `Array.prototype.sort`): find natural runs, reverse strictly descending ones, extend short runs to **minrun** (32 to 64 in practice) with binary insertion sort, then merge runs kept on a stack with balance rules. Merges use **galloping** when one run keeps winning.
- **Shell, Comb, Cocktail, Gnome and Odd-Even Sort Shell sort**: insertion sort on elements a gap apart, with gaps shrinking to 1. Knuth gaps 1, 4, 13, 40 ... give about $O(n^{1.5})$. In place, not stable, very little code.
- **Counting, Radix and Bucket Sort** Comparison sorts need at least

$$\log_2(n!) \approx n \log_2 n$$

comparisons. These sorts avoid the bound by using the values themselves as positions.
- **Cycle Sort and Pancake Sort Cycle sort** makes the minimum possible number of writes: every value is written at most once into its final position. Time $O(n^2)$, in place, not stable. Useful when writes are expensive, such as flash memory.
- **Choosing a Sorting Algorithm Small or nearly sorted**: insertion sort, which is why TimSort and introsort use it for small pieces.
- **Two Pointers Technique Opposite direction** (converging): `l = 0, r = n − 1`, move one or both inward each step. O(n) time, O(1) space.
- **Sliding Window Fixed window** of size k: keep a running sum. `sum += a[r] − a[r − k]`. Total O(n) instead of O(n·k).
- **Prefix Sums and Difference Arrays** Prefix array of length n + 1 with `P[0] = 0` and `P[i+1] = P[i] + a[i]`. Build O(n), each range-sum query O(1).
- **Frequency Arrays** When keys are small integers in a known range [0, K), an array of size K is a perfect hash table with no collisions.
- A dynamic programming idea: `current` is the best sum of a subarray that **ends exactly at i**.
- **Dutch National Flag** Edsger Dijkstra's three-way partition. One pass, O(n) time, O(1) space, in place.
- **Boyer-Moore Majority Vote Majority (> ⌊n/2⌋)**: one candidate and one counter. O(n) time, O(1) space.
- **Matrix Manipulations Rotate 90° clockwise in place**: transpose (`swap a[i][j], a[j][i]` for j > i), then reverse every row. $O(n^2)$ time, O(1) space.
- **Monotonic Stack and Queue** A **monotonic stack** keeps values strictly increasing or decreasing from bottom to top. Each index is pushed once and popped once, so O(n) total.
- **Two-Dimensional Prefix Sum** Build: `P[i+1][j+1] = a[i][j] + P[i][j+1] + P[i+1][j] − P[i][j]`. O(rows × cols) time and space.
- **Interval Overlapping Patterns Merge overlapping**: sort by start (O(n log n)), then one pass. If `start ≤ last.end`, set `last.end = max(last.end, end)`, else append.
- **Square Root Decomposition and Mo's Algorithm** Block size `b = ⌈√n⌉`. Precompute `block[k]` = sum of its b elements in O(n).
- **Fenwick Trees and Segment Trees** Both support **point update** and **range query** in **O(log n)**. A plain array is O(1) and O(n), a prefix array is O(n) and O(1).

TIPS AND TRAPS

REMEMBER — **Heap allocation**: memory comes from an allocator (C: `malloc`, Java/JS/Python: the runtime). It can be large and can outlive the function.
Lesson: Memory Allocation: Stack vs Heap

REMEMBER — In **Java** every array object lives on the heap. The variable `int[] a` is a reference stored in the stack frame.
Lesson: Memory Allocation: Stack vs Heap

REMEMBER — In **Python** and **JavaScript** lists and arrays are heap objects too. Names in a function only hold references.
Lesson: Memory Allocation: Stack vs Heap

REMEMBER Heap blocks are freed by `free` in C, or by the garbage collector once nothing refers to them.
Lesson: Memory Allocation: Stack vs Heap

REMEMBER The CPU does one multiply and one add, so access by index costs **O(1)** no matter how long the array is.
Lesson: Contiguous Memory Layout

REMEMBER Typical sizes: `byte` 1, `short` 2, `int` 4, `long` or `double` 8 bytes.
Lesson: Contiguous Memory Layout

REMEMBER Many CPUs have an addressing mode that does `base + index × scale` in a single instruction, with scale 1, 2, 4 or 8.
Lesson: Contiguous Memory Layout

REMEMBER Python's `array` module and JS typed arrays store raw values back to back like this. Python lists store references contiguously.
Lesson: Contiguous Memory Layout

REMEMBER Sequential array scans also trigger the **hardware prefetcher**, which loads the next lines before you ask.
Lesson: Cache Locality

REMEMBER A linked list node holds a pointer to the next node, which can be anywhere. Each hop may be a cache miss costing around 100 ns versus about 1 ns for an L1 hit.
Lesson: Cache Locality

REMEMBER Both scans are O(n) in Big-O terms, yet the array is often several times faster in practice.
Lesson: Cache Locality

REMEMBER The picture uses a small cache with LRU replacement (the oldest line is thrown out first).
Lesson: Cache Locality

REMEMBER **1-based**: Fortran, MATLAB, R, Lua, Julia. The compiler or runtime adjusts the offset.
Lesson: Array Indexing Mechanics

REMEMBER **Pointer arithmetic** (C): if `p` points to an int, `p + 1` moves 4 bytes, not 1. `a[i]` is defined as `*(a + i)`.
Lesson: Array Indexing Mechanics

REMEMBER Python also allows negative indexes: `a[-1]` is the last element, computed as `a[len(a) − 1]`.
Lesson: Array Indexing Mechanics

REMEMBER Java and Python check bounds and throw an error. C does not, which can silently read the wrong memory.
Lesson: Array Indexing Mechanics

REMEMBER **Runtime size**: the length comes from a variable (Java: `new int[n]`, C99 VLA). It is decided while running, then fixed forever.
Lesson: Static Arrays

REMEMBER New slots get a default value: `0` in Java for numbers, garbage in C for locals.
Lesson: Static Arrays

REMEMBER Writing past the end: Java throws `ArrayIndexOutOfBoundsException`, Python raises `IndexError`, a JS typed array silently ignores the write, C corrupts memory.
Lesson: Static Arrays

REMEMBER Benefits: no resize cost, predictable memory, best cache behaviour.
Lesson: Static Arrays

REMEMBER When full: allocate `capacity × growth`, copy `size` elements, free the old block. That push costs O(n).
Lesson: Dynamic Arrays and Amortized Cost

REMEMBER **Geometric growth** makes copies rare. With doubling, total copies for n pushes are below 2n, so the **amortized** cost per push is O(1).
Lesson: Dynamic Arrays and Amortized Cost

REMEMBER Growing by a fixed amount (say +10) instead gives $O(n^2)$ total, so O(n) per push.
Lesson: Dynamic Arrays and Amortized Cost

REMEMBER Real factors: Python list about 1.125, Java ArrayList 1.5, C++ vector 1.5 or 2.
Lesson: Dynamic Arrays and Amortized Cost

REMEMBER **Delete at i**: shift `a[i+1..n−1]` left by one, starting from i. Cost O(n − i − 1).
Lesson: Insertion, Deletion and Shifting

REMEMBER Insert or delete at the end is O(1). At the front is the worst case, O(n).
Lesson: Insertion, Deletion and Shifting

REMEMBER **Swap-with-last delete**: `a[i] = a[n−1]`, then `n−−`. O(1), but the order changes. Great for unordered bags of items such as game entities.
Lesson: Insertion, Deletion and Shifting

REMEMBER Loop in the same order as the layout. Walking a row-major matrix column by column jumps `cols × size` bytes each step and wastes cache lines.
Lesson: Multi-Dimensional Arrays

REMEMBER Java `int[][]` is really an array of row arrays (see Jagged Arrays). Flattening into one `int[rows*cols]` gives true contiguity.
Lesson: Multi-Dimensional Arrays

REMEMBER Going back: `row = k / cols`, `col = k % cols` for row-major.
Lesson: Multi-Dimensional Arrays

REMEMBER Java: `int[][] a = new int[3][]`, then `a[0] = new int[4]`. Python and JS: a list or array of lists.
Lesson: Jagged Arrays

REMEMBER Access `a[r][c]` takes two hops: load the row reference, then index into the row. Rows are not contiguous with each other.
Lesson: Jagged Arrays

REMEMBER Always check `c < a[r].length`, since every row has its own length.
Lesson: Jagged Arrays

REMEMBER Good for triangles (Pascal's triangle), adjacency lists, and ragged data.
Lesson: Jagged Arrays

REMEMBER **Best case** O(1): target at index 0. **Worst case** O(n): last or missing. **Average** about n/2 checks when present, so O(n).
Lesson: Linear Search

REMEMBER A normal loop does two tests per step: `i < n` and `a[i] == target`.
Lesson: Linear Search

REMEMBER **Sentinel search** saves the last element, writes the target there, and loops with only `a[i] != target`. Afterwards it restores the last element and checks whether the stop was real.
Lesson: Linear Search

REMEMBER The saving is a constant factor (about half the comparisons), not a change in Big-O.
Lesson: Linear Search

REMEMBER **Overflow bug**: `(low + high) / 2` can exceed the largest 32-bit int (2,147,483,647) and wrap to a negative number. `low + (high − low) / 2` never does. Java's own library had this bug until 2006.
Lesson: Binary Search

REMEMBER **Lower bound**: first index with `a[i] ≥ x`. **Upper bound**: first index with `a[i] > x`. Both use a half-open range `[low, high)` and can return n.
Lesson: Binary Search

REMEMBER Count of x = upperBound(x) − lowerBound(x).
Lesson: Binary Search

REMEMBER Python: `bisect_left`, `bisect_right`. Java: `Arrays.binarySearch`.
Lesson: Binary Search

REMEMBER **Bubble**: stable. With an early-exit flag it is O(n) on already sorted input.
Lesson: Bubble, Insertion and Selection Sort

REMEMBER **Insertion**: stable, adaptive, O(n + inversions). Best choice for small or nearly sorted arrays, so real libraries use it for tiny pieces.
Lesson: Bubble, Insertion and Selection Sort

REMEMBER **Selection**: not stable, always n(n−1)/2 comparisons, but at most n − 1 swaps. Useful when writes are expensive.
Lesson: Bubble, Insertion and Selection Sort

REMEMBER **Quick sort**: O(n log n) on average, in-place (O(log n) stack), not stable. Worst case $O(n^2)$ when pivots are always the smallest or largest.
Lesson: Merge Sort and Quick Sort

REMEMBER **Pivot strategies**: first or last element (bad on sorted input), middle, random (expected O(n log n) for any input), median of three (first, middle, last).
Lesson: Merge Sort and Quick Sort

REMEMBER This animation uses the **Lomuto** partition: the pivot is moved to the end, then values smaller than it are swapped to the front.
Lesson: Merge Sort and Quick Sort

REMEMBER Java sorts primitives with dual-pivot quick sort and objects with TimSort (merge-based). Python uses TimSort.
Lesson: Merge Sort and Quick Sort

REMEMBER **Build the max heap** bottom up: sift down every parent from

$$\lfloor n/2 \rfloor - 1$$

to 0. This costs only O(n), not O(n log n).
Lesson: Heap Sort

REMEMBER

Extract n − 1 times: swap the root with the last heap slot, shrink the heap, sift the new root down. Each sift is O(log n).

Lesson: Heap Sort

REMEMBER

Total O(n log n) in every case, O(1) extra space, **not stable**, and poor cache locality because it jumps around the array.

Lesson: Heap Sort

REMEMBER

Introsort falls back to heap sort to guarantee O(n log n), and heaps power priority queues and top k problems.

Lesson: Heap Sort

REMEMBER

TimSort is **stable**, O(n) on already sorted input, O(n log n) worst case, O(n) extra space.

Lesson: TimSort and Introsort (Hybrid Sorts)

REMEMBER

Introsort (C++ `std::sort`, .NET): quick sort with a recursion depth limit of about

$$2 \lfloor \log_2 n \rfloor$$

. Beyond the limit it heap sorts that piece. Pieces below about 16 values use insertion sort.

Lesson: TimSort and Introsort (Hybrid Sorts)

REMEMBER

Introsort is O(n log n) worst case, in place, and **not stable**.

Lesson: TimSort and Introsort (Hybrid Sorts)

REMEMBER

This animation uses a minrun of 4 and an insertion threshold of 4 so that every stage is visible on a short array, and merges neighbouring runs left to right instead of using TimSort's stack rules.

Lesson: TimSort and Introsort (Hybrid Sorts)

REMEMBER

Comb sort: bubble sort with a gap that shrinks by about 1.3 each pass. It kills **turtles** (small values near the end) quickly. Average close to O(n log n), worst O(n²).

Lesson: Shell, Comb, Cocktail, Gnome and Odd-Even Sort

REMEMBER

Cocktail shaker sort: bubble sort in both directions, so turtles also move fast. Stable, O(n²).

Lesson: Shell, Comb, Cocktail, Gnome and Odd-Even Sort

REMEMBER

Gnome sort: like insertion sort done with swaps only. Stable, O(n²), one loop and one index.

Lesson: Shell, Comb, Cocktail, Gnome and Odd-Even Sort

REMEMBER

Odd-even transposition sort: alternate passes compare pairs (1,2), (3,4) … and (0,1), (2,3) … Every pair in a pass is independent, so it runs well in parallel. n passes always suffice.

Lesson: Shell, Comb, Cocktail, Gnome and Odd-Even Sort

REMEMBER

Counting sort: O(n + k) time and space for values in a range of size k. With prefix sums and a right to left pass it is **stable**.

Lesson: Counting, Radix and Bucket Sort

REMEMBER

Radix sort (LSD): d passes of a stable counting sort on each digit, O(d × (n + b)) for base b. Stability of each pass is what makes it correct.

Lesson: Counting, Radix and Bucket Sort

REMEMBER

Bucket sort: O(n) on average when values are spread evenly, O(n²) if everything lands in one bucket.

Lesson: Counting, Radix and Bucket Sort

REMEMBER

They work for integers, fixed length keys and evenly spread numbers, not for arbitrary objects.

Lesson: Counting, Radix and Bucket Sort

REMEMBER The correct position of x is the number of values smaller than x, plus skips over equal values already placed.

Lesson: Cycle Sort and Pancake Sort

REMEMBER **Pancake sort** only reverses prefixes. It needs at most

$$2n - 3$$

flips. Each round finds the maximum of the unsorted prefix, flips it to the front, then flips it to its place.

Lesson: Cycle Sort and Pancake Sort

REMEMBER Prefix reversal shows up in genome rearrangement problems and in interview questions about sorting with a restricted operation.

Lesson: Cycle Sort and Pancake Sort

REMEMBER **General purpose**: quick sort or introsort in place, TimSort when stability matters.

Lesson: Choosing a Sorting Algorithm

REMEMBER **Guaranteed O(n log n) with O(1) space**: heap sort.

Lesson: Choosing a Sorting Algorithm

REMEMBER **Stable**: insertion, merge, TimSort, counting, radix. **Not stable**: selection, quick, heap, shell.

Lesson: Choosing a Sorting Algorithm

REMEMBER **Few writes**: selection sort (n − 1 swaps) or cycle sort (minimum writes).

Lesson: Choosing a Sorting Algorithm

REMEMBER **Small integer keys**: counting or radix sort, linear time.

Lesson: Choosing a Sorting Algorithm

REMEMBER Comparison counts are not the whole story: cache behaviour, branch prediction and memory allocation decide real speed, so always measure.

Lesson: Choosing a Sorting Algorithm

REMEMBER **Pair sum on a sorted array**: if `a[l] + a[r]` is too small, `l++`, too big, `r--`. Replaces the O(n²) double loop.

Lesson: Two Pointers Technique

REMEMBER **Container with most water**: area = `min(h[l], h[r]) × (r − l)`. Always move the shorter wall, because keeping it can never give a larger area.

Lesson: Two Pointers Technique

REMEMBER **Same direction** (fast and slow): `slow` marks the end of the kept part, `fast` scans. Used for dedupe, removing values, partitioning.

Lesson: Two Pointers Technique

REMEMBER **Variable window**: grow `r` to include more, then shrink `l` while the condition holds. Each index enters and leaves once, so O(n).

Lesson: Sliding Window

REMEMBER The shrinking trick needs a monotonic condition, such as all values being positive for a sum target.

Lesson: Sliding Window

REMEMBER Common uses: max sum of k items, smallest subarray with sum ≥ S, longest substring without repeats.

Lesson: Sliding Window

REMEMBER Works for any invertible operation: sums, XOR, counts.
Lesson: Prefix Sums and Difference Arrays

REMEMBER **Difference array** `D`: range update "add v to a[l..r]" is `D[l] += v`, `D[r+1] -= v`, so O(1) per update.
Lesson: Prefix Sums and Difference Arrays

REMEMBER After all updates, one prefix pass rebuilds the array: `a[i] += D[0] + … + D[i]`. Total O(n + q) instead of O(n·q).
Lesson: Prefix Sums and Difference Arrays

REMEMBER The two are inverses: the prefix sum of a difference array gives back the original.
Lesson: Prefix Sums and Difference Arrays

REMEMBER Update and lookup are O(1), memory is O(K). Much faster than a general hash map in practice.
Lesson: Frequency Arrays

REMEMBER For characters: `count[c - 'a']++`. For digits: `count[d]++`.
Lesson: Frequency Arrays

REMEMBER Uses: anagram checks, counting sort, finding the first unique character, histogram of grades.
Lesson: Frequency Arrays

REMEMBER Not a good fit when K is huge or keys are sparse, such as arbitrary 64-bit numbers.
Lesson: Frequency Arrays

REMEMBER O(n) time and O(1) space, compared with $O(n^2)$ for trying every start and end.
Lesson:

REMEMBER Start with `current = best = a[0]`, not 0. That way an all-negative array returns its largest single value.
Lesson:

REMEMBER To recover the indices, remember where the current run started, and copy it to the best range whenever `best` improves.
Lesson:

REMEMBER Invariant: `a[0..low-1] = 0`, `a[low..mid-1] = 1`, `a[mid..high]` unknown, `a[high+1..n-1] = 2`.
Lesson: Dutch National Flag

REMEMBER After swapping with `high`, do not move `mid`: the value that came back from high has not been checked yet.
Lesson: Dutch National Flag

REMEMBER The same idea powers 3-way quick sort, which handles many duplicate keys well.
Lesson: Dutch National Flag

REMEMBER **More than ⌊n/3⌋**: at most two such values can exist, so keep two candidates and two counters. A third different value cancels one from each.
Lesson: Boyer-Moore Majority Vote

REMEMBER The voting pass only finds candidates. A second pass must **verify** the counts, because no majority might exist.
Lesson: Boyer-Moore Majority Vote

REMEMBER Generalizes to **> ⌊n/k⌋** with k − 1 candidates (Misra-Gries).
Lesson: Boyer-Moore Majority Vote

REMEMBER **Rotate 90° counter-clockwise**: transpose, then reverse every column (or reverse rows first, then transpose).

Lesson: Matrix Manipulations

REMEMBER Index view: clockwise sends `(i, j)` to `(j, n − 1 − i)`.

Lesson: Matrix Manipulations

REMEMBER **Spiral order**: keep four borders `top, bottom, left, right` and shrink them after each side. Works for any rows × cols.

Lesson: Matrix Manipulations

REMEMBER **Diagonals**: main `i == j`, anti `i + j == n − 1`. Upper triangle `i < j`, lower `i > j`. In an odd-sized matrix the centre lies on both.

Lesson: Matrix Manipulations

REMEMBER **Next greater element**: keep a decreasing stack of indexes. While `a[top] < a[i]`, the answer for `top` is `a[i]`, pop it.

Lesson: Monotonic Stack and Queue

REMEMBER **Sliding window maximum**: a deque of indexes with decreasing values. Pop the back while it is ≤ the new value, pop the front when it leaves the window. The front is the maximum. O(n) instead of O(n·k).

Lesson: Monotonic Stack and Queue

REMEMBER Both are usually stored in a plain array with a top or head index.

Lesson: Monotonic Stack and Queue

REMEMBER Query any rectangle in **O(1)** with four lookups, using inclusion-exclusion.

Lesson: Two-Dimensional Prefix Sum

REMEMBER The extra row and column of zeros remove all edge-case checks.

Lesson: Two-Dimensional Prefix Sum

REMEMBER Used in image processing (integral images for fast box blur and face detection), game maps and matrix range queries.

Lesson: Two-Dimensional Prefix Sum

REMEMBER **Intersection of two sorted lists**: two pointers. Overlap is `[max(s1, s2), min(e1, e2)]` when that range is valid. Advance the interval that ends first. O(m + n).

Lesson: Interval Overlapping Patterns

REMEMBER Intervals here are closed: `[1, 3]` and `[3, 5]` touch, so they merge.

Lesson: Interval Overlapping Patterns

REMEMBER Related problems: insert interval, meeting rooms (min rooms via sorted starts and ends), non-overlapping intervals.

Lesson: Interval Overlapping Patterns

REMEMBER **Range query**: at most 2b loose elements plus n/b whole blocks, so O(√n).

Lesson: Square Root Decomposition and Mo's Algorithm

REMEMBER **Point update**: `block[i / b] += new − old`, O(1).

Lesson: Square Root Decomposition and Mo's Algorithm

REMEMBER **Mo's algorithm** (offline): sort queries by `([l / b], r)`, then move two pointers with add and remove steps. Total pointer movement is O((n + q)√n).

Lesson: Square Root Decomposition and Mo's Algorithm

REMEMBER Mo's shines when the answer is easy to update one element at a time but hard to combine, such as counting distinct values.

Lesson: Square Root Decomposition and Mo's Algorithm

REMEMBER **Fenwick (BIT)**: 1-indexed array `t`, where `t[i]` holds the sum of `(i − lowbit(i), i]` and `lowbit(i) = i & −i`.

Lesson: Fenwick Trees and Segment Trees

REMEMBER **Segment tree**: node k has children 2k and 2k+1 inside one array of size about 4n. Each node stores the sum of a range. Handles any associative operation (min, max, gcd) and lazy range updates.

Lesson: Fenwick Trees and Segment Trees

REMEMBER Query on a segment tree: a node fully inside the range returns its sum, a node outside returns 0, a partial node asks both children.

Lesson: Fenwick Trees and Segment Trees

2.3 Strings

Names, messages, web pages and DNA are all strings. This page shows what a string is underneath, why it never changes in place, and the scanning tricks that turn slow string code into fast code: two pointers, counting, sliding windows, KMP and rolling hashes.

KEY IDEAS

- **What a string really is** A string is a sequence of characters: a name, a sentence, a DNA sequence, a line of a log file. Underneath, the computer does not store letters at all. It stores numbers , one or more per character, and an encoding says which number means which character.
- **Immutable strings and building them fast** In Python, JavaScript and Java a string is immutable : once made, it never changes. Methods such as upper() or replace() return a new string and leave the old one alone.
- **Everyday operations and what they cost** Every language gives you a toolbox for strings. Knowing what each tool costs stops you from hiding an O(n) step inside a loop.
- **Two pointers: reverse and palindromes** Many string problems become easy with two indexes that walk toward each other: i from the left and j from the right. Each step moves at least one of them, so the whole walk is O(n) with O(1) extra space.
- **Counting characters: anagrams and frequency** Two words are anagrams when they use exactly the same letters the same number of times, like "listen" and "silent". Sorting both words and comparing works in O(n log n). Counting is faster.
- **Sliding window: the longest run without repeats** A window is a stretch of the string between two indexes, start and end . Instead of checking every one of the $n^2/2$ substrings, grow the window on the right one character at a time and shrink it on the left only when a rule is broken.
- **Pattern matching: naive search and KMP** Finding a pattern of length m inside a text of length n is one of the oldest string problems. Your editor's find box, grep and virus scanners all do it.
- **Rolling hash: Rabin and Karp** Another way to search is to give every window of the text a fingerprint , a number computed from its characters, and compare fingerprints instead of characters.

FORMULAS

Immutable strings and building them fast

Exam favourite

$$1 + 2 + 3 + \ldots + n = n(n + 1) / 2 \approx n^2 / 2$$

Lesson: Immutable strings and building them fast

Counting characters: anagrams and frequency

Exam favourite

index of a letter = code(letter) − code('a') for example code('c') − 97 = 99 − 97 = 2

Lesson: Counting characters: anagrams and frequency

Pattern matching: naive search and KMP

Exam favourite

$$\text{lps}[i] = \text{length of the longest proper prefix of } p[0..i] \text{ that is also a suffix of } p[0..i]$$

Lesson: Pattern matching: naive search and KMP

Rolling hash: Rabin and Karp

Exam favourite

$$\text{hash}(\text{window}) = (c_0 \cdot B^{(m-1)} + c_1 \cdot B^{(m-2)} + \ldots + c_{(m-1)}) \bmod M$$

Lesson: Rolling hash: Rabin and Karp

Rolling hash: Rabin and Karp

$$\text{next} = ((\text{hash} - c_{out} \cdot B^{(m-1)}) \cdot B + c_{in}) \bmod M$$

Lesson: Rolling hash: Rabin and Karp

TABLES

Table: encoding · what it covers · bytes per character

Encoding	What it covers	Bytes per character
ASCII	English letters, digits, punctuation (codes 0 to 127)	1
Unicode	Every writing system, emoji and symbol. Each character gets a **code point**.	depends on how it is stored
UTF-8	The most common way to store Unicode on disk and on the web	1 to 4
UTF-16	How Java and JavaScript store strings in memory	2 or 4

Lesson: What a string really is

Table: language · slow way · fast way

Language	Slow way	Fast way
Python	`s += x` in a loop	`"".join(pieces)`
JavaScript	`s += x` in a loop	`pieces.join("")`
Java	`s += x` in a loop	`StringBuilder.append(x)`

Lesson: Immutable strings and building them fast

Table: operation · python · javascript · java

Operation	Python	JavaScript	Java	Cost
Length	`len(s)`	`s.length`	`s.length()`	O(1)
Character at i	`s[i]`	`s[i]`	`s.charAt(i)`	O(1)
Slice from i to j	`s[i:j]`	`s.slice(i, j)`	`s.substring(i, j)`	O(j − i)
Find a piece	`s.find(p)`	`s.indexOf(p)`	`s.indexOf(p)`	O(n · m) worst case
Compare	`a == b, a < b`	`a === b, a < b`	`a.equals(b), a.compareTo(b)`	O(n)
Split and join	`s.split(" "), " ".join(w)`	`s.split(" "), w.join(" ")`	`s.split(" "), String.join(" ", w)`	O(n)
Replace	`s.replace(a, b)`	`s.replace(a, b)` (first match)	`s.replace(a, b)` (all matches)	O(n)

Lesson: Everyday operations and what they cost

Table: approach · idea · time

Approach	Idea	Time
Brute force	Check every substring for repeats	$O(n^3)$
Better brute force	Grow from each start until a repeat appears	$O(n^2)$
Sliding window	Each index enters and leaves the window at most once	O(n)

Lesson: Sliding window: the longest run without repeats

Table: i · 0 · 1 · 2

i	0	1	2	3	4
p[i]	a	b	a	b	c
lps[i]	0	0	1	2	0

Lesson: Pattern matching: naive search and KMP

Table: algorithm · preparation · search · shines when

Algorithm	Preparation	Search	Shines when
Naive	none	O(n · m) worst	short patterns, simple code
KMP	O(m) prefix table	O(n)	guaranteed linear time
Rabin and Karp	O(m) first hash	O(n + m) expected	many patterns at once, plagiarism checks

Lesson: Rolling hash: Rabin and Karp

TIPS AND TRAPS

COMMON MISTAKE An emoji such as 😀 is one character to you, but two UTF-16 units in Java and JavaScript, so `"😀".length` is 2 there and 1 in Python. Be careful when you count characters from user input.

Lesson: What a string really is

REMEMBER Changing one character of an immutable string means building a whole new string. That costs O(n), not O(1).

Lesson: Immutable strings and building them fast

IN THE EXAM In Java, == compares whether two variables point at the same object. Use `a.equals(b)` to compare the characters. In Python and JavaScript, == and === compare the characters.

Lesson: Immutable strings and building them fast

REMEMBER Slicing is not free. A loop that slices a long string on every turn can quietly turn an O(n) idea into $O(n^2)$. Move indexes instead, and slice once at the end.

Lesson: Everyday operations and what they cost

REMEMBER Counting shows up everywhere: the first character that does not repeat, the most common word, checking whether a ransom note can be cut out of a magazine. Whenever a problem says "same characters", think of a frequency table.

Lesson: Counting characters: anagrams and frequency

REMEMBER Both ends of a sliding window only ever move forward. That single fact is the proof that the loop is O(n).

Lesson: Sliding window: the longest run without repeats

STEP 3 Linear structures

3.1 Linked lists

A linked list is a chain of nodes joined by references. It gives up instant access by position, and in return inserting and deleting never move anything. This page builds lists by hand, rewires them, reverses and merges them, and shows the tortoise and hare trick that finds middles and loops.

KEY IDEAS

- **Nodes holding hands** A linked list is a chain of nodes . Each node holds a value and a reference (a pointer) to the next node. The list itself only remembers the first node, called the head . The last node points at nothing: None in Python, null in JavaScript and Java.
- **Inserting and deleting by rewiring** Nothing ever moves in a linked list. Inserting or deleting only changes a few references, which is why these operations can be O(1) once you are standing at the right place.
- **Doubly linked and circular lists** A doubly linked list gives every node two references: next and prev . It costs one more reference per node, and in return you can walk backwards and remove a node in O(1) when you hold that node, because it knows both of its neighbours.
- **Reversing a list in place** Reversing is the classic linked list exercise, and it trains the most important habit: save the next node before you change a reference .
- **Fast and slow pointers** Send two pointers down the list at different speeds: the tortoise moves one node per step and the hare moves two. This one idea answers several questions in a single pass with O(1) extra space.
- **Merging two sorted lists** Given two lists that are already sorted, build one sorted list by repeatedly taking the smaller front node. No new nodes are needed: we only rewire the existing ones. A dummy node again saves us from special cases at the start.
- **Lists versus arrays in real code** On paper, linked lists win at inserting and deleting. In real programs, arrays often win anyway, because their boxes sit next to each other and the CPU cache loads many of them at once. Following references jumps around memory.

FORMULAS

Fast and slow pointers Exam favourite

$$\text{gap after each step} = \text{gap} - 1 \text{ so they meet in at most (loop length) steps}$$

Lesson: Fast and slow pointers

TABLES

Table: operation · what changes · cost

Operation	What changes	Cost
Insert at head	`new.next = head`, then `head = new`	O(1)
Insert after a known node	`new.next = node.next`, then `node.next = new`	O(1)
Insert at tail	walk to the end first, or keep a `tail` reference	O(n), or O(1) with tail
Delete the node after prev	`prev.next = prev.next.next`	O(1)
Delete by value	walk to find the node before it, then unlink	O(n)

Lesson: Inserting and deleting by rewiring

Table: kind · references per node · walk backwards · remove a held node

Kind	References per node	Walk backwards	Remove a held node
Singly linked	1	no	O(n), you must find the node before it
Doubly linked	2	yes	O(1)
Circular	1 or 2	if doubly	same as its base kind

Lesson: Doubly linked and circular lists

Table: need · better choice · why

Need	Better choice	Why
Read item i often	array	O(1) access by index
Add or remove at both ends	deque or linked list	O(1) at the front and the back
Remove items you already hold, very often	doubly linked list	O(1) unlink without searching
Iterate over everything fast	array	cache friendly, no pointer chasing
Least recently used cache	hash map + doubly linked list	O(1) lookup and O(1) move to front

Lesson: Lists versus arrays in real code

TIPS AND TRAPS

REMEMBER You can only enter a singly linked list through its head and only move forward. Lose the head and the whole list is gone.

Lesson: Nodes holding hands

COMMON MISTAKE Order matters. When inserting after a node, set `new.next` first. If you set `node.next = new` first, the rest of the list is lost.

Lesson: Inserting and deleting by rewiring

REMEMBER The recursive version is shorter but uses one stack frame per node, so it needs O(n) space and can overflow the call stack on a list of a million nodes.

Lesson: Reversing a list in place

IN THE EXAM To find where the loop **starts**, put one pointer back at the head after they meet and move both one step at a time. They meet again exactly at the start of the loop.

Lesson: Fast and slow pointers

3.2 Stacks and queues

Two of the simplest structures in computing, and two of the most used. A stack hands back the newest item first, a queue hands back the oldest. This page builds both, then uses them to match brackets, evaluate expressions, run ring buffers, find next greater elements and power undo and redo.

KEY IDEAS

- **The stack: last in, first out** A stack is a pile. You add to the top (push), look at the top (peek) and remove from the top (pop). The last item in is the first item out, which is called LIFO .
- **Matching brackets with a stack** Code editors, compilers and HTML checkers all need to know whether brackets are properly nested. A stack does it in one pass: push every opener, and when a closer arrives, the top of the stack must be its partner.
- **Evaluating expressions** People write infix expressions like (3 + 4) * 2 − 5 , with operators between the numbers and parentheses for order. Computers prefer postfix (also called reverse Polish notation), where each operator comes after its two operands: 3 4 + 2 * 5 − .
- **The queue: first in, first out** A queue is a line at a ticket counter. Items join at the back (enqueue) and leave from the front (dequeue). The first item in is the first item out: FIFO .

- **Deques and priority queues** A heap is a complete binary tree stored in an array: the children of index i sit at 2i + 1 and 2i + 2, and every parent is smaller than its children. The Trees topic explains it in detail.
- **Monotonic stacks** A monotonic stack keeps its items in increasing or decreasing order. It answers "what is the next bigger (or smaller) item to my right?" for every position in a single pass.
- **Undo, redo and other everyday uses** Undo and redo are two stacks. Every change pushes the old state onto the undo stack. Undo moves the current state to the redo stack and pops the previous one. A brand new change clears the redo stack, because the future it remembered no longer exists.

FORMULAS

The queue: first in, first out Exam favourite

$$\text{tail} = (\text{head} + \text{size}) \bmod \text{capacity} \quad \text{head} = (\text{head} + 1) \bmod \text{capacity}$$

Lesson: The queue: first in, first out

Monotonic stacks Exam favourite

$$\text{pushes} + \text{pops} \le n + n = 2n$$

Lesson: Monotonic stacks

TABLES

Table: operation · python list · javascript array · java arraydeque

Operation	Python list	JavaScript array	Java `ArrayDeque`	Cost
push	`s.append(x)`	`s.push(x)`	`s.push(x)`	O(1) amortized
pop	`s.pop()`	`s.pop()`	`s.pop()`	O(1)
peek	`s[-1]`	`s[s.length - 1]`	`s.peek()`	O(1)
is empty	`not s`	`s.length === 0`	`s.isEmpty()`	O(1)

Lesson: The stack: last in, first out

Table: token · action · stack after

Token	Action	Stack after
3, 4	push	3 4
+	pop 4 and 3, push 7	7
2	push	7 2
*	pop 2 and 7, push 14	14
5	push	14 5
–	pop 5 and 14, push 9	9

Lesson: Evaluating expressions

Table: · python · javascript · java

	Python	JavaScript	Java
Deque	`collections.deque`	none built in	`ArrayDeque`
Priority queue	`heapq` on a list	none built in, write a small heap	`PriorityQueue`

Lesson: Deques and priority queues

Table: where · structure · why

Where	Structure	Why
Browser back button	stack	the most recent page comes back first
Printer jobs, chat messages	queue	fair order of arrival
Breadth first search	queue	visit nearest places first
Depth first search, backtracking	stack (or recursion)	finish the newest path first
Operating system scheduler	priority queue	important tasks run first
Recent items, sliding windows	deque	add on one end, drop from the other

Lesson: Undo, redo and other everyday uses

TIPS AND TRAPS

COMMON MISTAKE In Java, avoid the old `Stack` class. It is synchronized and slower. `ArrayDeque` is the recommended stack and queue.

Lesson: The stack: last in, first out

REMEMBER Whenever the most recent unfinished thing must be finished first, think stack: brackets, nested tags, undo, function calls, backtracking.

Lesson: Matching brackets with a stack

IN THE EXAM Pop order matters for − and /. The first pop gives the **right** operand. For 8 2 − you pop 2, then 8, and compute 8 − 2 = 6, not 2 − 8.

Lesson: Evaluating expressions

REMEMBER Keyboard input, audio streaming and network cards all use ring buffers: a fixed block of memory that is reused forever, with O(1) enqueue and dequeue.

Lesson: The queue: first in, first out

COMMON MISTAKE Removing the first element of a Python list (`pop(0)`) or a JavaScript array (`shift()`) moves every other element, which is **O(n)**. Use `collections.deque` in Python, `ArrayDeque` in Java, and a head index or a ring buffer in JavaScript.

Lesson: The queue: first in, first out

REMEMBER The same pattern solves "days until a warmer day", stock spans, the largest rectangle in a histogram and trapping rain water.

Lesson: Monotonic stacks

STEP 4 Trees and graphs

4.1 Trees

Trees organise data as a hierarchy, from one root down to many leaves. This page covers the vocabulary and the recursive pattern behind almost every tree problem, four traversal orders, binary search trees with insert, search and delete, why balance matters, heaps stored in arrays, and tries for words.

KEY IDEAS

- **What a tree is** A tree stores items in a hierarchy. One node at the top is the root . Every other node has exactly one parent , and any node can have children below it. A node with no children is a leaf .
- **Traversals: visiting every node** A traversal visits every node once. Depth first traversals go deep before going wide, and differ only in when the node itself is visited compared with its two subtrees.
- **Binary search trees** A binary search tree (BST) adds one rule: everything in the left subtree of a node is smaller than the node, and everything in the right subtree is larger. That rule turns every node into a signpost.

- **Deleting from a search tree** Deleting is the only tricky BST operation, because the tree must keep its ordering rule. First find the node, then handle one of three cases.
- **Balance: why shape matters** A BST is only fast when it is short . Insert values that are already sorted and every new node becomes a right child, so the tree turns into a chain that is no better than a linked list.
- **Heaps: trees inside arrays** A binary heap is a complete binary tree: every level is full except possibly the last, which fills from the left. In a min heap every parent is smaller than or equal to its children, so the smallest value is always at the root.
- **Tries: trees of letters** A trie (also called a prefix tree) stores words letter by letter. Each edge is one letter, and words that share a beginning share the same path. A flag on a node marks where a complete word ends.

FORMULAS

🔢 Edges

Exam favourite

$$\text{edges} = n - 1$$

Lesson: What a tree is

📏 Levels

$$n \le 2^h - 1$$

Lesson: What a tree is

Balance: why shape matters

Exam favourite

$$h = \lceil \log_2(n + 1) \rceil$$

Lesson: Balance: why shape matters

Heaps: trees inside arrays

Exam favourite

$$\text{left} = 2i + 1 \quad \text{right} = 2i + 2 \quad \text{parent} = \lfloor (i - 1) / 2 \rfloor$$

Lesson: Heaps: trees inside arrays

TABLES

Table: word · meaning · in the tree below

Word	Meaning	In the tree below
root	the only node with no parent	1
children	nodes directly below a node	2 and 3 are children of 1
leaf	a node with no children	4, 5 and 6
depth	edges from the root down to a node	depth of 5 is 2
height	number of levels in the tree	3
subtree	a node together with everything below it	2, 4, 5

Lesson: What a tree is

Table: order · rule · result for the tree · typical use

Order	Rule	Result for the tree	Typical use
Preorder	node, left, right	1 2 4 5 3 6	copying or saving a tree
Inorder	left, node, right	4 2 5 1 3 6	sorted output from a search tree
Postorder	left, right, node	4 5 2 6 3 1	deleting a tree, folder sizes
Level order	row by row with a queue	1 2 3 4 5 6	shortest number of steps, printing by level

Lesson: Traversals: visiting every node

Table: operation · balanced tree · worst case (a chain)

Operation	Balanced tree	Worst case (a chain)
search	O(log n)	O(n)
insert	O(log n)	O(n)
delete	O(log n)	O(n)
sorted listing	O(n)	O(n)
minimum or maximum	O(log n)	O(n)

Lesson: Binary search trees

Table: self balancing tree · idea · used in

Self balancing tree	Idea	Used in
AVL tree	heights of the two subtrees differ by at most 1, fixed with rotations	lookup heavy in memory indexes
Red black tree	colour rules keep the longest path at most twice the shortest	Java `TreeMap`, C++ `std::map`
B tree	many keys per node, all leaves on one level	databases and file systems

Lesson: Balance: why shape matters

Table: · heap · binary search tree

	Heap	Binary search tree
Smallest value	O(1) peek	O(log n)
Insert	O(log n)	O(log n) when balanced
Search any value	O(n)	O(log n) when balanced
Storage	one array	nodes and references

Lesson: Heaps: trees inside arrays

TIPS AND TRAPS

REMEMBER Most tree code follows one pattern: handle the empty tree, solve the left subtree, solve the right subtree, then combine the two answers with the current node.

Lesson: What a tree is

REMEMBER Every traversal takes O(n) time. Recursion uses O(h) memory for the call stack, where h is the height. Level order uses memory for the widest level.

Lesson: Traversals: visiting every node

IN THE EXAM Inorder traversal of a BST always produces the values in sorted order. Exams love asking for it, and it is also the quickest way to check that a tree really is a BST.

Lesson: Binary search trees

REMEMBER The inorder predecessor (the largest value in the left subtree) works just as well. Both are the closest values to the one being removed, so the order is kept.

Lesson: Deleting from a search tree

COMMON MISTAKE Python and JavaScript have no built in balanced search tree. For sorted keys in Python, the `bisect` module on a sorted list or the third party `sortedcontainers` package is the usual answer.

Lesson: Balance: why shape matters

REMEMBER Python has `heapq`, Java has `PriorityQueue`. Heaps power priority queues, heap sort, Dijkstra's shortest paths and finding the top k items.

Lesson: Heaps: trees inside arrays

REMEMBER Choose a trie when the questions are about prefixes. Choose a hash set when you only ever ask whether a whole word exists.

Lesson: Tries: trees of letters

4.2 Graphs

Graphs model anything made of things and links: roads, friendships, networks and plans. This page shows how to store a graph, explore it with breadth first and depth first search, put tasks in a valid order, find the cheapest routes with Dijkstra, connect everything with a minimum spanning tree, and treat grids as graphs.

KEY IDEAS

- **What a graph is** A graph is a set of vertices (also called nodes) joined by edges . Unlike a tree, a graph has no root and may contain loops, so you can walk in a circle and come back to where you started.
- **Breadth first search** Breadth first search (BFS) explores a graph in rings. It visits the start, then every vertex one edge away, then every vertex two edges away, and so on. A queue keeps the discovered vertices in order, and a seen mark stops any vertex from being added twice.
- **Depth first search** Depth first search (DFS) follows one path as far as it can go, then backs up to the last vertex with an unexplored neighbour and tries again. Recursion gives it a stack for free. Switch the animation above to DFS to compare the two orders.
- **Topological order** Some tasks must happen before others: socks before shoes, a course before the next course, a library before the program that uses it. Draw an arrow from each task to the task that depends on it, and you get a directed acyclic graph (DAG).
- **Dijkstra's shortest paths** When edges have weights, the path with the fewest edges is not always the cheapest. Dijkstra's algorithm grows a set of settled vertices whose shortest distance is final.
- **Minimum spanning trees and union find** A town wants to connect every house to the network with the least cable. A spanning tree connects all V vertices with exactly V − 1 edges and no cycle. The minimum spanning tree (MST) is the one with the smallest total weight.
- **Grids are graphs too** Many puzzles hide a graph inside a grid. Every cell is a vertex, and it is joined to its four neighbours up, down, left and right. You never build an adjacency list. You compute the neighbours with small offsets and check that they stay inside the grid.

FORMULAS

🤝 Handshakes Exam favourite

$$\text{sum of all degrees} = 2E$$

Lesson: What a graph is

🔢 Most possible edges

$$E \le V(V-1) / 2$$

Lesson: What a graph is

Breadth first search Exam favourite

$$O(V + E)$$

Lesson: Breadth first search

Dijkstra's shortest paths Exam favourite

$$\text{if } dist[v] + w < dist[u] \text{ then } dist[u] = dist[v] + w$$

Lesson: Dijkstra's shortest paths

Dijkstra's shortest paths

$$O((V + E) \log V)$$

Lesson: Dijkstra's shortest paths

TABLES

Table: · adjacency list · adjacency matrix

	Adjacency list	Adjacency matrix
Idea	for each vertex, a list of its neighbours	a V × V grid with 1 where an edge exists
Memory	O(V + E)	$O(V^2)$
Is u next to v?	O(degree of u)	O(1)
Visit all neighbours of u	O(degree of u)	O(V)
Best for	sparse graphs, which is most real graphs	small or very dense graphs

Lesson: What a graph is

Table: · bfs · dfs

	BFS	DFS
Helper structure	queue	stack or recursion
Explores	ring by ring	one deep path at a time
Shortest path without weights	yes	no
Good for	fewest steps, nearest first	components, cycles, mazes, topological order
Time	O(V + E)	O(V + E)

Lesson: Depth first search

Table: step · cost

Step	Cost
sort the edges	O(E log E)
find and union, with path compression	almost O(1) each
total	O(E log E)

Lesson: Minimum spanning trees and union find

TIPS AND TRAPS

REMEMBER In a graph without weights, the first time BFS reaches a vertex is along a shortest path. Remember the parent of each vertex and you can walk the path back.

Lesson: Breadth first search

COMMON MISTAKE Deep recursion can overflow the call stack on a graph with hundreds of thousands of vertices in a line. Python stops at about 1000 levels by default. For large inputs, write DFS with your own stack.

Lesson: Depth first search

IN THE EXAM If the order ends up shorter than the number of vertices, some vertices never became ready. That only happens when the graph has a **cycle**, so Kahn's algorithm is also a cycle detector.

Lesson: Topological order

COMMON MISTAKE Dijkstra needs weights that are zero or positive. A negative edge can make a settled distance wrong. For negative weights use Bellman-Ford, which relaxes every edge V − 1 times in O(V · E).

Lesson: Dijkstra's shortest paths

REMEMBER Union find also answers "are these two computers on the same network?" and "which edge closes a cycle?" without a full search.

Lesson: Minimum spanning trees and union find

REMEMBER

Scan every cell. When you find land that has not been seen, it starts a new island, and a BFS or DFS flood marks the whole island so it is never counted again. Each cell is handled a constant number of times, so the total is O(rows × cols).

Lesson: Grids are graphs too